Java OOP MCQ Topics

> topics i need cover to answer any mcq questions from java oops

To answer **any MCQ from Java OOPs**, you need to cover **these topics completely**. This is an **exam-oriented checklist**—if you know everything here, MCQs won't trick you.

1️⃣ Basics of OOP

Understand **why OOP exists** and its core ideas.

- What is **OOP**?
- Benefits of OOP
  - Reusability
  - Modularity
  - Maintainability
  - Security
- Difference between **OOP and Procedural Programming**
- Object vs Class
- Real-world examples of OOP

2️⃣ Class & Object (VERY IMPORTANT)

MCQs often target syntax & behavior.

- What is a **class**
- What is an **object**
- Creating objects using `new`
- Multiple objects from same class
- Instance variables vs Local variables
- Default values of instance variables
- Memory allocation (Heap vs Stack – basic idea)

3️⃣ Encapsulation

Frequently asked conceptually and via code.

- What is **encapsulation**

- How encapsulation is achieved
  - private variables
  - public getters/setters
- Advantages of encapsulation
- Data hiding vs Encapsulation
- Can a class be fully encapsulated?
- Real-life example

4️⃣ Inheritance (🔥 MCQ Favorite)

Maximum MCQs come from here.

Types of Inheritance in Java

- Single
- Multilevel
- Hierarchical
- Why **multiple inheritance not supported** with classes
- Multiple inheritance using **interfaces**

Keywords & Rules

- `extends`
- `super`
- Constructor inheritance
- Order of constructor execution
- `super()` vs `this()`

Inheritance Rules

- Private members inheritance
- Protected access
- Can constructors be inherited? ❌
- Can static methods be inherited? ✅
- Can final methods be inherited? ❌

5️⃣ Polymorphism (🔥🔥 VERY IMPORTANT)

Almost guaranteed MCQs.

Types

- Compile-time polymorphism
  - Method overloading
- Runtime polymorphism
  - Method overriding

Method Overloading

- Same method name, different parameters
- Return type alone cannot overload
- Access modifier doesn't matter
- Static method overloading
- Constructor overloading

Method Overriding

- Same method signature
- Access level rules
- `@Override` annotation
- Overriding static methods ❌ (method hiding)
- Overriding final methods ❌
- Overriding private methods ❌
- Covariant return type

6️⃣ Abstraction

MCQs often confuse abstraction with encapsulation.

Abstract Class

- `abstract` keyword
- Abstract vs concrete methods
- Can abstract class have:
  - constructors ✅
  - variables ✅
  - non-abstract methods ✅
- Can we create object of abstract class? ❌

Interface

- What is an interface
- `implements` keyword
- Multiple inheritance using interface
- Interface variables (public static final by default)
- Interface methods (public abstract by default)

Java 8+ Interface Features

- default methods
- static methods
- private methods (Java 9+)

Abstract Class vs Interface (VERY IMPORTANT)

7️⃣ Access Modifiers (🔥 MCQ Trap Area)

You must know **scope clearly**.

Modifier	Same Class	Same Package	Subclass	Outside
private	✅	❌	❌	❌
default	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

- Default access (no keyword)
- Protected behavior with inheritance

8️⃣ Constructors

MCQs test execution order & rules.

- What is a constructor
- Default constructor
- Parameterized constructor
- Constructor overloading
- Constructor chaining
- `this()` vs `super()`
- Can constructor be:
  - static ❌

  - final ❌
  - abstract ❌
- Return type of constructor (none)

9️⃣ `this` Keyword

Small topic but MCQ favorite.

- Refers to current object
- Used to:
  - differentiate variables
  - call current class method
  - call constructor (`this()`)
- Cannot be used in static context

🔟 `super` Keyword

Often mixed with `this`.

- Refers to parent class object
- Used to:
  - access parent variables
  - call parent methods
  - call parent constructor
- `super()` must be first statement

1️⃣1️⃣ Static Keyword

Very high-weight MCQ topic.

- Static variable
- Static method
- Static block
- Static vs non-static
- Can static method access non-static data? ❌
- Static method overriding ❌
- Static members memory location

1️⃣2️⃣ Final Keyword

Easy MCQs but tricky wording.

- final variable (constant)
- final method (cannot override)
- final class (cannot inherit)
- Blank final variable
- Can constructor be final? ❌

1️⃣3️⃣ Object Class (Often Ignored but Important)

MCQs test inherited methods.

- `toString()`
- `equals()`
- `hashCode()`
- `getClass()`
- `clone()`
- `finalize()`
- All classes implicitly extend `Object`

1️⃣4️⃣ Relationship Between Classes

Conceptual MCQs.

- IS-A relationship (Inheritance)
- HAS-A relationship (Aggregation / Composition)
- Difference between Aggregation & Composition

1️⃣5️⃣ Exception Basics (OOP Related MCQs)

Not full exception handling—only OOP angle.

- Exception vs Error
- Checked vs Unchecked
- Exception hierarchy
- `throw` vs `throws`
- Method overriding & exceptions rule

1️⃣6️⃣ Java Memory Concepts (Basic)

MCQs often mix memory + OOP.

- Stack vs Heap
- Object creation memory
- Reference variables
- Garbage Collection basics

1️⃣7️⃣ Common MCQ Traps You MUST Know

- Can we overload `main()`? ✅
- Can we override `main()`? ❌
- Can interface have constructor? ❌
- Can abstract class be final? ❌
- Can a class be both abstract and final? ❌
- Can we create object of interface? ❌

✅ If you want:

- 📌 **MCQ practice questions** from each topic
- 📌 **One-page OOPs revision sheet**
- 📌 **Trick MCQs with explanations**

Tell me and I'll prepare them **exam-ready** for you.

> lets start with Basics of OOP
> prepare me to answer any mcq question there can be from this subtopic

Perfect 👍
We'll go **step-by-step** and make you **MCQ-proof**.

Below is **EVERYTHING you must know** from **Basics of OOP**, explained in **simple exam language**, with **traps + MCQ angles**.

🔰 BASICS OF OOP (Object-Oriented Programming)

1️⃣ What is OOP?

Definition (Exam-ready):
Object-Oriented Programming (OOP) is a programming approach where a program is built using

objects that represent **real-world entities** and contain **data + behavior**.

👉 MCQ keyword:

- Objects
- Classes
- Real-world modeling
- Data + methods together

2️⃣ Why OOP is needed? (Purpose)

OOP is used to solve problems of **procedural programming**.

Problems with Procedural Programming:

- No proper data security
- Difficult to manage large programs
- Code reusability is poor
- Changes affect whole program

👉 MCQ trap:
❌ OOP is NOT for faster execution
❌ OOP does NOT reduce memory usage

3️⃣ Key Features / Principles of OOP (🔥 VERY IMPORTANT)

These **4 pillars** appear in almost every exam.

1. Encapsulation

- Wrapping **data + methods** into one unit (class)
- Achieved using **access modifiers**

2. Inheritance

- One class acquires properties of another class
- Achieves **code reuse**

3. Polymorphism

- One thing, many forms
- Method overloading & overriding

4. Abstraction

- Showing **essential details**
- Hiding **implementation**

👉 MCQ trick:

- Data hiding ≠ Abstraction
- Data hiding is achieved using **encapsulation**

4️⃣ Class

Definition:
A **class** is a **blueprint/template** from which objects are created.

Characteristics:

- Logical entity
- Does NOT occupy memory
- Contains:
  - Variables (fields)
  - Methods
  - Constructors

👉 MCQ traps:

- ❌ Class is not a real-world entity
- ❌ Class does not hold data at runtime

5️⃣ Object

Definition:
An **object** is an **instance of a class**.

Characteristics:

- Real-world entity
- Occupies memory
- Created using `new` keyword

👉 Example:

```java
Car c = new Car();
```

👉 MCQ traps:

- Object is created at runtime
- Multiple objects can be created from one class
- Object has **state & behavior**

6️⃣ Difference Between Class and Object (🔥 HIGH MCQ CHANCE)

Class	Object
Blueprint	Instance
Logical entity	Real entity
No memory	Occupies memory
Defined once	Created multiple times

7️⃣ Real-world Entity vs OOP Entity

Real-world entity → OOP representation

Real World	OOP
Student	Class
Name, age	Variables
Study(), sleep()	Methods

👉 MCQ angle:
Objects represent **real-world entities**

8️⃣ OOP vs Procedural Programming (🔥)

Procedural	OOP
Function-based	Object-based
Data is exposed	Data is hidden
Poor security	High security
No reusability	Reusable code

👉 MCQ trap:

- C is procedural
- Java is object-oriented
- C++ supports both

9️⃣ Advantages of OOP (EXAM FAVORITE)

✔️ Code Reusability
✔️ Security
✔️ Easy Maintenance
✔️ Modularity
✔️ Scalability

👉 MCQ trick:
❌ Faster execution is NOT an advantage
❌ Less memory usage is NOT guaranteed

🔟 Disadvantages of OOP (Sometimes asked)

- Complex design
- More memory usage
- Slower than procedural (in some cases)

1️⃣1️⃣ Is Java 100% Object-Oriented? (🔥🔥 VERY COMMON MCQ)

Answer: ❌ NO

Why?

Because Java has:

- Primitive data types (`int`, `float`, etc.)
- Static methods
- `main()` method

👉 MCQ twist:

- Java is **mostly object-oriented**, not fully

1️⃣2️⃣ Important MCQ One-liners (MEMORIZE)

- OOP is based on **objects**
- Class is a **logical entity**
- Object is a **physical entity**
- Objects are created using **new**
- OOP improves **security & reusability**
- In OOP, data and methods are **tightly bound**

1️⃣3️⃣ Common MCQ Traps (🔥 DO NOT FALL)

Question	Correct
OOP reduces execution time	❌
Class occupies memory	❌
Object occupies memory	✅
OOP uses real-world concepts	✅
Java is fully OOP	❌

1️⃣4️⃣ 5 QUICK MCQs (Test Yourself)

1. OOP stands for?
   A. Object Oriented Process
   B. Object Oriented Programming ✅
   C. Object Operating Program
2. Which is NOT a feature of OOP?
   A. Encapsulation
   B. Inheritance
   C. Compilation ❌
3. Which occupies memory?
   A. Class
   B. Object ✅
4. Which is a blueprint?
   A. Object
   B. Class ✅
5. Java is:
   A. Fully OOP
   B. Mostly OOP ✅

✅ You are now MCQ-ready for Basics of OOP

Next options:

👉 **Class & Object (deep MCQ level)**
👉 **4 Pillars of OOP (separate deep dive)**
👉 **Trick MCQs from Basics of OOP**

Tell me what to continue with 🔥

now Class & Object (deep MCQ level)

Perfect 🔥
This is one of the **highest-weight MCQ areas in Java OOP**, so we'll go **deep + exam-oriented**.

Below is **EVERYTHING about Class & Object** that examiners use to **trap students**.

🚀 CLASS & OBJECT (DEEP MCQ LEVEL)

1️⃣ What is a Class? (MCQ Definition)

Class is a **user-defined data type** that acts as a **blueprint** for creating objects.

Key Points (MEMORIZE)

- Logical entity
- Does **NOT** occupy memory
- Defined using `class` keyword
- Can contain:
  - Variables
  - Methods
  - Constructors
  - Blocks

👉 MCQ trap:
❌ Class is not an object
❌ Class does not store data at runtime

2️⃣ Syntax of Class

```
java

class Student {
    int id;
    String name;

    void study() {
        System.out.println("Studying");
    }
}
```

👉 MCQ:

- Class name must start with **capital letter** (convention, not rule)
- Only **one public class per file**
- File name = public class name

3️⃣ What is an Object?

Object is an **instance of a class** that represents a **real-world entity**.

Key Characteristics

- Created at runtime
- Occupies memory in **heap**
- Has:
  - State (variables)
  - Behavior (methods)
  - Identity (reference)

4️⃣ Object Creation Process (🔥 MCQ FAVORITE)

```java
Student s = new Student();
```

Steps involved:

1. Class is loaded
2. Memory allocated in heap
3. Constructor is called
4. Reference variable stores address

👉 MCQ trap:

- `new` allocates memory
- Constructor initializes object
- Reference is stored in stack

5️⃣ Reference Variable

- Stores address of object
- Can point to only one object at a time
- Multiple references can point to same object

```java
Student s1 = new Student();
Student s2 = s1;
```

👉 MCQ:

- Object is not duplicated
- Both refer to same memory

6️⃣ Object Without Reference (Anonymous Object)

```
java

new Student().study();
```

- Used only once
- No reference stored
- Eligible for garbage collection

👉 MCQ:
❌ Cannot reuse anonymous object

7️⃣ Multiple Objects from Same Class

```
java

Student s1 = new Student();
Student s2 = new Student();
```

👉 MCQ:

- Each object has separate memory
- Changes in one object do NOT affect another

8️⃣ Instance Variables vs Local Variables (🔥)

Instance Variables

- Declared inside class
- Have default values
- Stored in heap

Local Variables

- Declared inside method
- No default value
- Stored in stack

👉 MCQ trap:
Using local variable without initialization → ❌ Compile-time error

---

9️⃣ Default Values of Instance Variables (🔥 VERY IMPORTANT)

Data Type	Default Value
int	0
float	0.0
double	0.0
char	'\u0000'
boolean	false
String / Object	null

👉 MCQ:
Local variables do NOT get default values

🔟 Method Inside Class

```java
void display() {
    System.out.println("Hello");
}
```

Method types:

- Instance method
- Static method

👉 MCQ:

- Instance method needs object to call
- Static method does NOT need object

1️⃣1️⃣ Calling Methods

```java
s.study();        // instance method
Student.info();  // static method
```

👉 MCQ trap:
Static method cannot directly access instance variables

1️⃣2️⃣ Constructor (INTRO LEVEL for MCQs)

- Same name as class
- No return type
- Called automatically
- Used to initialize object

```
java

Student() {
    System.out.println("Constructor called");
}
```

👉 MCQ:

- Constructor is not a method
- Can be overloaded
- Cannot be static

1️⃣3️⃣ Memory Allocation (🔥 EXAM FAVORITE)

Stack Memory

- Reference variables
- Local variables
- Method calls

Heap Memory

- Objects
- Instance variables

👉 MCQ trick:

- Object always lives in heap
- Reference may live in stack or heap

1️⃣4️⃣ Accessing Variables (VERY IMPORTANT)

```
java

Student s = new Student();
s.id = 10;
```

👉 MCQ:

- Variables accessed using **dot operator**
- `.` is member access operator

1️⃣5️⃣ this Keyword (MCQ Angle)

- Refers to current object
- Used to differentiate instance variable & local variable

```java
this.id = id;
```

👉 MCQ:
Cannot use **`this`** in static context

1️⃣6️⃣ Can a Class Have Multiple Objects? ✅

YES.

👉 MCQ:

- One class → many objects
- One object → one memory block

1️⃣7️⃣ Can Object Exist Without Class? ❌

NO.

👉 MCQ trap:

- Class can exist without object
- Object cannot exist without class

1️⃣8️⃣ Can We Create Object Without `new`? (ADVANCED MCQ)

YES, using:

- **`clone()`**
- Reflection
- Deserialization
- Factory methods

👉 MCQ:
Most common way → **`new`**

1️⃣9️⃣ Garbage Collection (Object MCQs)

- Object becomes eligible when no reference exists
- Java handles GC automatically

```
java

s = null;
```

2️⃣0️⃣ Common MCQ Traps (🔥🔥)

Statement	True / False
Class occupies memory	❌
Object created at compile time	❌
Constructor returns value	❌
Instance variable gets default value	✅
Local variable gets default value	❌

2️⃣1️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Which keyword creates object?
   👉 **new**
2. Memory for object is allocated in?
   👉 **Heap**
3. Which does NOT have default value?
   👉 **Local variable**
4. How many objects can be created from one class?
   👉 **Any number**
5. Reference variable stores?
   👉 **Address of object**
6. Object is created when?
   👉 **Runtime**
7. Which is a logical entity?
   👉 **Class**
8. Which can exist without object?
   👉 **Class**
9. Object without reference is called?
   👉 **Anonymous object**

10. `this` refers to?
    👉 **Current object**

✅ You are now MCQ-ready for Class & Object

What next?

🔥 **Constructors (deep MCQ + traps)**
🔥 **Encapsulation (MCQ level)**
🔥 **Inheritance (maximum MCQs)**

Tell me which one to continue with 👌

> now lets go with Constructors (deep MCQ + traps)

Excellent choice 🔥
Constructors are a **top MCQ trap area**—questions look simple but options are designed to confuse.

Below is a **COMPLETE, deep, exam-ready guide** to **Constructors in Java**, covering **every MCQ angle**.

🚀 CONSTRUCTORS IN JAVA (DEEP MCQ + TRAPS)

1️⃣ What is a Constructor?

Definition (Exam-ready):
A **constructor** is a **special member of a class** that is **automatically invoked when an object is created** and is used to **initialize the object**.

2️⃣ Key Characteristics (🔥 MEMORIZE)

- Name **must be same as class name**
- **No return type** (not even `void`)
- Called **automatically**
- Used to initialize instance variables
- Executes **once per object**

👉 MCQ trap:
❌ Constructor is NOT a method
❌ Constructor does NOT return value

3️⃣ Syntax of Constructor

```
java

class Student {
    Student() {
        System.out.println("Constructor called");
    }
}
```

4️⃣ Types of Constructors (🔥)

1. Default Constructor

- Provided by compiler **only if no constructor is defined**
- Initializes variables with default values

👉 MCQ trap:

- If you write **any constructor**, compiler will NOT provide default constructor

---

2. No-Argument Constructor

```
java

Student() {
}
```

- Written by programmer
- Same as default but user-defined

---

3. Parameterized Constructor

```
java

Student(int id, String name) {
    this.id = id;
    this.name = name;
}
```

👉 MCQ:
Used to initialize object with **custom values**

5️⃣ Constructor Overloading (🔥 VERY IMPORTANT)

Multiple constructors with:

- Same name

- Different parameters

```
java

Student() {}
Student(int id) {}
Student(int id, String name) {}
```

👉 MCQ trap:

- Return type does NOT matter
- Parameter list must differ

6️⃣ Constructor vs Method (🔥 MCQ FAVORITE)

Constructor	Method
Same name as class	Any name
No return type	Has return type
Called automatically	Called manually
Initializes object	Performs operation

7️⃣ Constructor Invocation Order (🔥🔥)

Case: Inheritance

- Parent constructor executes first
- Then child constructor

```
java

class A {
    A() { System.out.println("A"); }
}
class B extends A {
    B() { System.out.println("B"); }
}
```

👉 Output:

```
css

A
B
```

8️⃣ this() vs super() (🔥🔥 VERY IMPORTANT)

this()

- Calls **current class constructor**
- Must be **first statement**
- Used in constructor chaining

```
java

this(10);
```

super()

- Calls **parent class constructor**
- Must be **first statement**
- Inserted automatically if not written

👉 MCQ traps:
❌ Cannot use both `this()` and `super()` together
❌ Cannot place after first line

9️⃣ Constructor Chaining (🔥)

Calling one constructor from another.

- Same class → `this()`
- Parent class → `super()`

🔟 Access Modifiers in Constructors

Constructors can be:

- public
- protected
- default
- private

👉 MCQ:

- Private constructor used in **Singleton pattern**

1️⃣1️⃣ Can Constructor be Inherited? ❌

NO.

👉 MCQ trap:

- Parent constructor is **invoked**, not inherited

1️⃣2️⃣ Can Constructor be Overridden? ❌

NO.

👉 Reason:

- Constructors are not methods

1️⃣3️⃣ Can Constructor be Overloaded? ✅

YES.

1️⃣4️⃣ Can Constructor be Static? ❌

NO.

1️⃣5️⃣ Can Constructor be Final? ❌

NO.

1️⃣6️⃣ Can Constructor be Abstract? ❌

NO.

1️⃣7️⃣ Can Constructor Have Access Modifiers? ✅

YES.

1️⃣8️⃣ Can Constructor Throw Exception? (MCQ TRAP)

YES.

```
java

Student() throws Exception {
}
```

👉 Rule:

- Child constructor must handle or declare parent exception

1️⃣9️⃣ Implicit super() Call (🔥 VERY COMMON MCQ)

If you don't write `super()`:

- Java inserts it automatically
- Calls parent no-arg constructor

👉 MCQ trap:
If parent has **no no-arg constructor**, compilation fails

2️⃣0️⃣ Default Constructor & Inheritance (🔥)

```java
class A {
    A(int x) {}
}
class B extends A {
}
```

❌ Compile-time error
👉 Because parent has no no-arg constructor

2️⃣1️⃣ Object Creation & Constructor Execution (🔥)

- Each object → constructor executes once
- Multiple objects → constructor executes multiple times

2️⃣2️⃣ Memory & Constructor

- Constructor does NOT create object
- `new` creates object
- Constructor initializes object

2️⃣3️⃣ Common MCQ Traps Table (🔥🔥)

Statement	True/False
Constructor has return type	❌
Constructor is method	❌
Constructor can be overloaded	✅
Constructor can be inherited	❌
Constructor can be private	✅
Constructor creates object	❌

2️⃣4️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. What is used to initialize object?
   👉 Constructor
2. Constructor is called when?
   👉 Object is created
3. Return type of constructor?
   👉 None
4. Can constructor be static?
   👉 No
5. Can constructor be overloaded?
   👉 Yes
6. Which executes first?
   👉 Parent constructor
7. Keyword to call current constructor?
   👉 this()
8. Keyword to call parent constructor?
   👉 super()
9. Default constructor provided when?
   👉 No constructor defined
10. Private constructor is used in?
    👉 Singleton pattern

✅ You are now MCQ-PROOF in Constructors

What next?

🔥 **Encapsulation (deep MCQ)**
🔥 **Inheritance (MAX MCQs + traps)**
🔥 **Polymorphism (overloading vs overriding MCQs)**

Tell me the next topic and we'll continue 💪

> now encapsulation(deep mcq level)

Great 👍
Encapsulation is a **classic MCQ trap topic** because questions mix it with **abstraction, data hiding, access modifiers**, and **getters/setters**.

Below is a **COMPLETE, deep, exam-ready guide** so you can answer **any MCQ** from encapsulation.

🔐 ENCAPSULATION IN JAVA (DEEP MCQ LEVEL)

1️⃣ What is Encapsulation?

Definition (Exam-ready):
Encapsulation is the process of **wrapping data (variables) and methods into a single unit (class)** and **restricting direct access to data**.

👉 Keyword to remember:

- Binding data + methods
- Security
- Controlled access

2️⃣ How Encapsulation is Achieved in Java (🔥 VERY IMPORTANT)

Encapsulation is achieved by:

1. Declaring variables as **private**
2. Providing **public getter and setter methods**

```
java

class Student {
    private int id;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }
}
```

👉 MCQ trap:
❌ Encapsulation is NOT achieved by abstraction
❌ Encapsulation does NOT require interface

3️⃣ Why Encapsulation is Needed?

- Protects data from unauthorized access
- Improves security
- Improves maintainability
- Allows validation of data

- Supports flexibility

👉 MCQ trick:
Encapsulation helps **data hiding**, but they are **not same**

4️⃣ Encapsulation vs Data Hiding (🔥 VERY COMMON MCQ)

Encapsulation	Data Hiding
Wrapping data + methods	Restricting access
Concept	Result
Achieved using class	Achieved using access modifiers

👉 One-liner:
Encapsulation leads to data hiding

5️⃣ Fully Encapsulated Class (🔥)

A class is **fully encapsulated** when:

- All variables are **private**
- Accessed only through **public getters/setters**

👉 Example:

```
java

class Account {
    private double balance;
}
```

👉 MCQ:

- Fully encapsulated class = maximum security

6️⃣ Access Modifiers Role in Encapsulation (🔥)

Encapsulation mainly uses:

- **`private`** → hide data
- **`public`** → controlled access

👉 MCQ trap:

- **`protected`** and **`default`** provide **partial encapsulation**

7️⃣ Advantages of Encapsulation (🔥 MCQ FAVORITE)

✔️ Data security
✔️ Control over data
✔️ Flexibility
✔️ Reusability
✔️ Easy maintenance

👉 MCQ trap:
❌ Encapsulation does NOT increase execution speed

8️⃣ Disadvantages of Encapsulation (Sometimes asked)

- Slight performance overhead
- More code (getters/setters)

9️⃣ Can Methods be Encapsulated? (MCQ)

YES.

👉 Explanation:

- Methods are encapsulated inside class
- Access controlled via access modifiers

🔟 Can We Achieve Encapsulation Without Getter/Setter? (🔥 MCQ TRAP)

YES.

Example:

```
java


private int id;
public void show() { }
```

👉 Getter/setter are **common**, not mandatory

1️⃣1️⃣ Encapsulation vs Abstraction (🔥🔥 VERY IMPORTANT)

Encapsulation	Abstraction
How things are grouped	What to show
Data protection	Hiding implementation
Achieved using class	Achieved using abstract class/interface

👉 MCQ trap:

- Encapsulation ≠ Abstraction

1️⃣2️⃣ Encapsulation vs Inheritance (MCQ)

Encapsulation	Inheritance
Security	Reusability
Private variables	Parent-child relationship
Data protection	Code reuse

1️⃣3️⃣ Can Encapsulation Exist Without Inheritance? ✅

YES.

👉 MCQ:

- Encapsulation is independent of inheritance

1️⃣4️⃣ Is Encapsulation a Compile-time or Runtime Concept?

👉 **Compile-time**

Reason:

- Access modifiers are checked at compile time

1️⃣5️⃣ Encapsulation & Immutability (ADVANCED MCQ)

- Immutable class uses encapsulation
- No setters provided

Example: **`String`** class

1️⃣6️⃣ Common MCQ Traps (🔥🔥)

Statement	True / False
Encapsulation hides data	✅
Encapsulation hides implementation	❌
Data hiding uses access modifiers	✅

Statement	True / False
Getter/setter mandatory	❌
Fully encapsulated class has public variables	❌

1️⃣7️⃣ One-Line MCQ Answers (MEMORIZE)

- Encapsulation binds data and methods
- Data hiding is achieved using **`private`**
- Getter returns value
- Setter sets value
- Fully encapsulated class → all variables private
- Encapsulation improves security

1️⃣8️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Encapsulation means?
   👉 Wrapping data and methods
2. Which keyword supports encapsulation most?
   👉 private
3. Fully encapsulated class contains?
   👉 Only private variables
4. Getter method is used to?
   👉 Read data
5. Setter method is used to?
   👉 Modify data
6. Encapsulation improves?
   👉 Security
7. Data hiding is achieved using?
   👉 Access modifiers
8. Encapsulation is checked at?
   👉 Compile time
9. Encapsulation requires inheritance?
   👉 No
10. Which OOP feature protects data?
    👉 Encapsulation

✅ You are now MCQ-READY in Encapsulation

What should we do next?

🔥 **Inheritance (deep MCQ + output-based traps)**
🔥 **Polymorphism (overloading vs overriding MCQs)**
🔥 **Abstraction (abstract class vs interface traps)**

Tell me the next topic and we'll continue 🚀

> now polymorphism(deep mcq based concepts)

Excellent 🔥
Polymorphism is one of the **TOP MCQ + output-based question topics** in Java OOP.
Examiners **love confusing options** here.

Below is a **COMPLETE, deep, MCQ-focused guide**—after this, **no polymorphism MCQ can trick you**.

🔁 POLYMORPHISM IN JAVA (DEEP MCQ CONCEPTS)

1️⃣ What is Polymorphism?

Definition (Exam-ready):
Polymorphism means **one name, many forms**—the same method behaves differently in different situations.

👉 MCQ keywords:

- One interface, many implementations
- Same method name, different behavior

2️⃣ Types of Polymorphism in Java (🔥 VERY IMPORTANT)

1. Compile-Time Polymorphism

- Achieved using **Method Overloading**
- Also called **Static Polymorphism**

2. Runtime Polymorphism

- Achieved using **Method Overriding**
- Also called **Dynamic Polymorphism**

👉 MCQ trap:

- Constructor overloading → Compile-time polymorphism

- Operator overloading → ❌ NOT supported (except +)

3️⃣ Compile-Time Polymorphism (Method Overloading)

Definition

Multiple methods with:

- Same name
- Different parameter list
- Same class

```
java

void add(int a, int b) {}
void add(int a, int b, int c) {}
```

4️⃣ Overloading Rules (🔥 MCQ FAVORITE)

✔️ Parameter count must differ
✔️ Parameter type must differ
✔️ Order of parameters can differ

❌ Return type alone cannot overload
❌ Access modifier does NOT matter

5️⃣ Valid & Invalid Overloading (MCQ Traps)

❌ Invalid

```
java

int sum(int a, int b) {}
double sum(int a, int b) {} // ❌
```

✅ Valid

```
java

int sum(int a, int b) {}
double sum(double a, double b) {}
```

6️⃣ Method Overloading & Type Promotion (🔥🔥)

Java follows this order:

```
arduino

byte → short → int → long → float → double
```

👉 MCQ:

- Exact match preferred
- Widening allowed
- Autoboxing last
- Varargs last

7️⃣ Can We Overload main()? (🔥)

YES.

```
java

public static void main(String[] args) {}
public static void main(int a) {}
```

👉 MCQ trap:

- JVM calls only **`String[]`** version

8️⃣ Can Constructors be Overloaded? ✅

YES → Compile-time polymorphism

9️⃣ Runtime Polymorphism (Method Overriding)

Definition

A child class provides **specific implementation** of parent method.

```
java

class A {
    void show() {}
}
class B extends A {
    void show() {}
}
```

🔟 Overriding Rules (🔥🔥 VERY IMPORTANT)

✔️ Same method name
✔️ Same parameters

✔️ IS-A relationship
✔️ Runtime decision

❌ Cannot override static method
❌ Cannot override final method
❌ Cannot override private method

1️⃣1️⃣ Method Overriding & Access Modifiers (🔥)

- Access level cannot be **more restrictive**
- Can be same or more permissive

```
java

protected → public ✅
public → protected ❌
```

1️⃣2️⃣ Static Methods & Polymorphism (🔥 MCQ TRAP)

- Static methods are **NOT overridden**
- This is called **method hiding**

👉 MCQ:

- Static method call depends on reference type

1️⃣3️⃣ Final Methods & Polymorphism

- Final method ❌ cannot be overridden
- Used to prevent runtime polymorphism

1️⃣4️⃣ Private Methods & Polymorphism

- Private methods ❌ cannot be overridden
- They are not visible to child class

1️⃣5️⃣ Method Overloading vs Overriding (🔥🔥)

Overloading	Overriding
Same class	Parent-child
Compile-time	Runtime
Parameter list differs	Same parameters

Overloading	Overriding
Return type may differ	Covariant return allowed

1️⃣6️⃣ Runtime Polymorphism & Reference (🔥🔥 VERY IMPORTANT)

```
java

A obj = new B();
obj.show();
```

👉 Output depends on:
✔️ Object type
❌ Reference type

1️⃣7️⃣ Upcasting & Downcasting (MCQ)

- Upcasting → Parent reference to child object (safe)
- Downcasting → Needs explicit cast

```
java

A a = new B(); // upcasting
B b = (B) a;  // downcasting
```

1️⃣8️⃣ Can Constructors be Overridden? ❌

NO.

1️⃣9️⃣ Can Static Methods be Overloaded? ✅

YES.

2️⃣0️⃣ Covariant Return Type (🔥)

Child method can return:

- Same return type
- Child of return type

2️⃣1️⃣ Common MCQ Traps Table (🔥🔥)

Statement	True/False
Polymorphism achieved by overloading	✅
Polymorphism achieved by overriding	✅
Static method supports polymorphism	❌
Final method supports polymorphism	❌
Private method supports polymorphism	❌

2️⃣2️⃣ Output-Based MCQ Example (🔥🔥)

```java
class A {
    void show() { System.out.println("A"); }
}
class B extends A {
    void show() { System.out.println("B"); }
}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        obj.show();
    }
}
```

👉 Output:

```css
B
```

2️⃣3️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Polymorphism means?
   👉 One name many forms
2. Method overloading is?
   👉 Compile-time polymorphism
3. Method overriding is?
   👉 Runtime polymorphism
4. Static method overriding?
   👉 Not possible
5. Which supports polymorphism?
   👉 Method overriding

6. main() can be overloaded?
   👉 Yes
7. Constructor overriding?
   👉 Not possible
8. Method selection in runtime polymorphism depends on?
   👉 Object type
9. Private method overridden?
   👉 No
10. Final method overridden?
    👉 No

✅ You are now MCQ-PROOF in Polymorphism

What next?

🔥 **Inheritance (deep MCQ + constructor + super traps)**
🔥 **Abstraction (abstract class vs interface MCQs)**
🔥 **Access Modifiers (high-trap MCQs)**

Tell me the next topic and we'll continue 🚀

> Abstration(deep mcq level)

Perfect 🔥
Abstraction is a **high-trap MCQ topic** because questions deliberately mix it with **encapsulation, interfaces, abstract classes, Java 8+ rules**, and **real-life logic**.

Below is a **COMPLETE, deep, exam-ready guide** so you can answer **ANY MCQ from Abstraction**.

🎭 ABSTRACTION IN JAVA (DEEP MCQ LEVEL)

1️⃣ What is Abstraction?

Definition (Exam-ready):
Abstraction is the process of **hiding implementation details** and **showing only essential features** to the user.

👉 Keywords examiners look for:

- Hide implementation
- Show what, not how
- Reduce complexity

2️⃣ Why Abstraction is Needed?

- Simplifies complex systems
- Improves security
- Reduces code dependency
- Helps achieve loose coupling

👉 MCQ trap:
❌ Abstraction does NOT hide data
❌ Abstraction does NOT improve execution speed

3️⃣ How Abstraction is Achieved in Java (🔥 VERY IMPORTANT)

Java supports abstraction using:

1. **Abstract class**
2. **Interface**

👉 MCQ:

- Access modifiers alone ❌ do NOT achieve abstraction

4️⃣ Abstract Class (🔥🔥 VERY IMPORTANT)

Definition

A class declared using **`abstract`** keyword.

```
java

abstract class Vehicle {
    abstract void run();
}
```

5️⃣ Abstract Class Rules (MCQ GOLD)

✔️ Can have abstract and non-abstract methods
✔️ Can have constructors
✔️ Can have instance variables
✔️ Can have static methods
❌ Cannot be instantiated

6️⃣ Abstract Methods

- Declared without body
- Must be implemented by child class

```java
abstract void display();
```

👉 MCQ trap:

- Abstract method cannot be private
- Abstract method cannot be static
- Abstract method cannot be final

7️⃣ Can Abstract Class Have Constructor? (🔥 VERY COMMON MCQ)

👉 **YES**

Purpose:

- Initialize variables
- Called via **`super()`**

8️⃣ Can Abstract Class Have main() Method?

👉 **YES**

9️⃣ Can Abstract Class Be Final? ❌

Reason:

- Abstract → needs inheritance
- Final → prevents inheritance

🔟 Can Abstract Class Have No Abstract Methods? ✅

YES.

```java
abstract class Test {
    void show() {}
}
```

👉 MCQ trap:

- Such class still cannot be instantiated

1️⃣1️⃣ Interface (🔥🔥 VERY IMPORTANT)

Definition

A **blueprint of a class** that contains **abstract methods**.

```
java

interface Animal {
    void sound();
}
```

1️⃣2️⃣ Interface Rules (MCQ FAVORITE)

✔️ All methods are public by default
✔️ Variables are public static final
✔️ Supports multiple inheritance
❌ Cannot create object
❌ Cannot have constructor

1️⃣3️⃣ Implementing Interface

```
java

class Dog implements Animal {
    public void sound() {
        System.out.println("Bark");
    }
}
```

👉 MCQ:

- **`implements`** keyword used

1️⃣4️⃣ Java 8+ Interface Features (🔥🔥🔥 MCQ TRAP)

✔️ Default methods
✔️ Static methods
✔️ Private methods (Java 9+)

```
java

default void show() {}
static void info() {}
```

👉 MCQ:

- Default method CAN be overridden
- Static interface method CANNOT be overridden

1️⃣5️⃣ Abstract Class vs Interface (🔥🔥 VERY IMPORTANT)

Abstract Class	Interface
Uses **`abstract`**	Uses **`interface`**
Can have constructors	No constructors
Can have variables	Only constants
Single inheritance	Multiple inheritance
Can have concrete methods	Mostly abstract

1️⃣6️⃣ Abstraction vs Encapsulation (🔥 MCQ FAVORITE)

Abstraction	Encapsulation
Hides implementation	Hides data
What to show	How to protect
Achieved by abstract/interface	Achieved by class

1️⃣7️⃣ Can We Achieve Abstraction Without Abstract Class? ✅

YES → using **interface**

1️⃣8️⃣ Can Abstract Method Be Overridden? ✅

YES.

1️⃣9️⃣ Abstract Class & Inheritance

- Child must implement all abstract methods
- Or child must also be abstract

2️⃣0️⃣ Real-Life Example (MCQ Concept)

ATM Machine:

- What user sees → Withdraw, Deposit
- How it works → Hidden

👉 MCQ:
This represents **abstraction**

2️⃣1️⃣ Common MCQ Traps (🔥🔥)

Statement	True/False
Abstract class can have constructor	✅
Abstract class can be instantiated	❌
Interface supports multiple inheritance	✅
Interface variables are mutable	❌
Abstract method can be private	❌

2️⃣2️⃣ One-Line Answers (MEMORIZE)

- Abstraction hides implementation
- Abstract class cannot be instantiated
- Interface variables are constants
- Default methods introduced in Java 8
- Abstract class supports inheritance

2️⃣3️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Abstraction means?
   👉 Hiding implementation
2. Which keyword is used for abstraction?
   👉 abstract
3. Which supports multiple inheritance?
   👉 Interface
4. Can abstract class have constructor?
   👉 Yes
5. Can we create object of abstract class?
   👉 No
6. Can interface have variables?
   👉 Yes (public static final)

7. Java 8 feature in interface?
   👉 Default methods
8. Abstract method body?
   👉 Not allowed
9. Abstract class without abstract method?
   👉 Allowed
10. Interface constructor?
    👉 Not allowed

✅ You are now MCQ-PROOF in Abstraction

What next?

🔥 **Inheritance (deep MCQ + super + constructor traps)**
🔥 **Access Modifiers (high-trap MCQs)**
🔥 **Object Class methods (equals, hashCode MCQs)**

Tell me what to continue with 🚀

> Inheritance (deep MCQ + super + constructor traps)

Excellent 🔥
Inheritance is a **TOP-WEIGHT MCQ topic** because examiners mix **types, access modifiers, constructors, `super`, overriding rules, and outputs**.

Below is a **COMPLETE, deep, MCQ-PROOF guide**. If you master this, **inheritance questions become free marks**.

🧬 INHERITANCE IN JAVA (DEEP MCQ + SUPER + CONSTRUCTOR TRAPS)

1️⃣ What is Inheritance?

Definition (Exam-ready):
Inheritance is the mechanism by which **one class acquires the properties and methods of another class**.

👉 Keywords:

- Code reusability
- IS-A relationship
- Parent–child relationship

2️⃣ Basic Syntax

```java
class Parent {
    int x = 10;
}

class Child extends Parent {
    int y = 20;
}
```

👉 MCQ:

- **`extends`** keyword used
- Child gets access to parent members (not all)

3️⃣ Types of Inheritance in Java (🔥 VERY IMPORTANT)

Supported:

1. **Single**
2. **Multilevel**
3. **Hierarchical**

Not Supported (with classes):

❌ Multiple inheritance
❌ Hybrid inheritance

👉 MCQ trap:

- Multiple inheritance supported using **interfaces**

4️⃣ IS-A Relationship (🔥)

Inheritance represents **IS-A** relationship.

Example:

- Dog IS-A Animal
- Car IS-A Vehicle

👉 MCQ:

- HAS-A ≠ inheritance

5️⃣ Members Inherited & NOT Inherited (🔥🔥)

Inherited:

✔️ public members
✔️ protected members
✔️ default members (same package)

NOT Inherited:

❌ private members
❌ constructors

👉 MCQ trap:

- Constructors are **invoked**, not inherited

6️⃣ Access Modifiers & Inheritance (🔥🔥 VERY IMPORTANT)

Modifier	Same Class	Same Package	Subclass (diff pkg)
private	✅	❌	❌
default	✅	✅	❌
protected	✅	✅	✅
public	✅	✅	✅

👉 MCQ:

- **`protected`** allows access in subclass even in different package

---

7️⃣ super Keyword (🔥🔥🔥 MCQ FAVORITE)

Uses of `super`:

1. Access parent variables
2. Call parent methods
3. Call parent constructor

8️⃣ super.variable (MCQ)

```java

class A {
    int x = 10;
}
class B extends A {
    int x = 20;
    void show() {
        System.out.println(super.x);
```

```
    }
}
```

👉 Output:

```
10
```

9️⃣ super.method() (MCQ)

```java
class A {
    void show() { System.out.println("A"); }
}
class B extends A {
    void show() {
        super.show();
        System.out.println("B");
    }
}
```

👉 Output:

```
A
B
```

🔟 super() Constructor Call (🔥🔥🔥)

- Calls **parent class constructor**
- Must be **first statement**
- Automatically inserted if not written

```java
class A {
    A() { System.out.println("A"); }
}
class B extends A {
    B() { System.out.println("B"); }
}
```

👉 Output:

```
A
B
```

1️⃣1️⃣ Constructor Execution Order (🔥🔥)

1. Parent constructor
2. Child constructor

Always.

1️⃣2️⃣ Parent Has No No-Arg Constructor (🔥 MCQ TRAP)

```
java

class A {
    A(int x) {}
}
class B extends A {
    B() {}
}
```

❌ Compile-time error

👉 Fix:

```
java

B() {
    super(10);
}
```

1️⃣3️⃣ this() vs super() (🔥🔥)

this()	super()
Same class constructor	Parent constructor
Used for constructor chaining	Used for inheritance
First statement	First statement

❌ Cannot use both together

1️⃣4️⃣ Method Overriding in Inheritance (🔥)

- Same method signature
- Child provides implementation

👉 Runtime polymorphism depends on **object**, not reference

1️⃣5️⃣ Overriding Rules with Inheritance (🔥🔥)

✔️ Access modifier cannot be more restrictive
✔️ Return type can be covariant
❌ static methods cannot be overridden
❌ final methods cannot be overridden
❌ private methods cannot be overridden

1️⃣6️⃣ Variable Hiding vs Method Overriding (🔥 MCQ TRAP)

```
java

class A {
    int x = 10;
}
class B extends A {
    int x = 20;
}
A obj = new B();
System.out.println(obj.x);
```

👉 Output:

```
10
```

Reason:

- Variables are resolved by **reference type**
- Methods by **object type**

1️⃣7️⃣ Static Members & Inheritance (🔥)

- Static methods are inherited
- But cannot be overridden
- This is called **method hiding**

1️⃣8️⃣ Final Class & Inheritance (🔥)

```
java

final class A {}
class B extends A {} // ❌
```

👉 MCQ:

- Final class cannot be inherited

1️⃣9️⃣ Abstract Class & Inheritance (🔥)

- Abstract class must be inherited
- Child must implement abstract methods

2️⃣0️⃣ Multiple Inheritance & Diamond Problem (🔥🔥)

Java avoids:

- Ambiguity
- Diamond problem

👉 Solved using:

- Interfaces (with default methods + override)

2️⃣1️⃣ Object Creation & Inheritance (🔥 MCQ)

- Object creation always creates **full object**
- Parent part + child part

2️⃣2️⃣ Common MCQ Traps Table (🔥🔥🔥)

Statement	True / False
Constructors are inherited	❌
Parent constructor executes first	✅
super() can be second statement	❌
Private members inherited	❌
Multiple inheritance supported via class	❌
Variables support polymorphism	❌

2️⃣3️⃣ Output-Based MCQ (🔥🔥)

```
java


class A {
    A() { System.out.print("A "); }
}
class B extends A {
    B() { System.out.print("B "); }
}
class C extends B {
    C() { System.out.print("C "); }
}
public class Test {
    public static void main(String[] args) {
        new C();
```

```
    }
}
```

👉 Output:

```
css

A B C
```

2️⃣4️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Inheritance represents?
   👉 IS-A relationship
2. Keyword used?
   👉 extends
3. Which inheritance not supported by class?
   👉 Multiple
4. Constructors are?
   👉 Not inherited
5. Parent constructor called using?
   👉 super()
6. Which executes first?
   👉 Parent constructor
7. super() must be?
   👉 First statement
8. Private members inherited?
   👉 No
9. Static method overridden?
   👉 No
10. Final class inherited?
    👉 No

✅ You are now MCQ-PROOF in Inheritance

What next?

🔥 **Access Modifiers (most confusing MCQs)**
🔥 **this vs super (mixed MCQs)**
🔥 **Object class methods (equals, hashCode, toString)**

Tell me the next topic and we'll finish the full OOP MCQ coverage 💪

Access Modifiers (most confusing MCQs)

Perfect choice 🔥
Access Modifiers are **one of the MOST CONFUSING MCQ areas** in Java because questions mix **packages, inheritance, protected behavior, and default access**.

Below is a **COMPLETE, deep, MCQ-PROOF guide**. After this, you won't get trapped.

🔐 ACCESS MODIFIERS IN JAVA (MOST CONFUSING MCQs)

1️⃣ What are Access Modifiers?

Definition (Exam-ready):
Access modifiers define the **visibility and accessibility** of **classes, variables, methods, and constructors**.

👉 Keywords:

- Scope
- Visibility
- Access control

2️⃣ Types of Access Modifiers (🔥 MUST MEMORIZE)

Java has **4 access levels**:

1. **`private`**
2. **`default`** (no keyword)
3. **`protected`**
4. **`public`**

3️⃣ Access Level Table (🔥🔥🔥 VERY IMPORTANT)

Modifier	Same Class	Same Package	Subclass (diff pkg)	Anywhere
private	✅	❌	❌	❌
default	✅	✅	❌	❌
protected	✅	✅	✅	❌
public	✅	✅	✅	✅

👉 **MEMORIZE THIS TABLE** – 70% MCQs come from here.

4️⃣ private (🔥 MCQ TRAPS)

Rules:

- Accessible **only within same class**
- Not accessible outside class (even subclass)
- Used for data hiding

```
java

private int x;
```

👉 MCQ traps:

- ❌ private members are NOT inherited
- ❌ private methods cannot be overridden

5️⃣ default (Package-Private) (🔥)

- No keyword used
- Accessible within **same package only**
- NOT accessible in subclass of different package

👉 MCQ:

- Default ≠ public

6️⃣ protected (🔥🔥🔥 MOST CONFUSING)

Rules:

- Accessible within same package
- Accessible in subclass outside package
- But **ONLY through inheritance**

👉 MCQ TRAP CODE:

```
java

// pkg1
class A {
    protected int x = 10;
}

// pkg2
class B extends A {
    void show() {
        System.out.println(x); // ✅
```

```
    }
}
```

❌ But this fails:

```
java

A a = new A();
System.out.println(a.x); // ❌ outside package
```

👉 **Golden Rule**:

> Protected outside package → ONLY via subclass reference

7️⃣ public (🔥)

- Accessible everywhere
- Least restrictive

👉 MCQ:

- Public members are inherited

8️⃣ Access Modifiers with Class (🔥 MCQ)

Top-level classes can be:

- **`public`**
- **`default`**

❌ Cannot be:

- private
- protected

9️⃣ Access Modifiers with Constructors (🔥)

- Constructors can be private, default, protected, public
- Private constructor → Singleton pattern

🔟 Access Modifiers & Methods (🔥)

- Methods follow same rules as variables
- Overriding rule:
  - Cannot reduce visibility

```
java

protected → public ✅
public → protected ❌
```

1️⃣1️⃣ Access Modifiers & Variables (🔥)

- Instance variables usually private
- Static variables follow same rules

1️⃣2️⃣ Access Modifiers & Inheritance (🔥🔥)

Member Type	Inherited?
private	❌
default	✅ (same pkg)
protected	✅
public	✅

1️⃣3️⃣ Access Modifiers & Overriding (🔥🔥)

Parent	Child Allowed
public	public
protected	protected / public
default	default / protected / public
private	Not overridden

1️⃣4️⃣ Protected vs Default (🔥🔥 MOST CONFUSED)

protected	default
Same pkg	Same pkg
Subclass (diff pkg)	❌
Supports inheritance	Limited

👉 MCQ:

- protected is **less restrictive** than default

1️⃣5️⃣ Access Modifiers & Interfaces (🔥 MCQ)

- Interface methods → **public by default**
- Interface variables → **public static final**

❌ Cannot use private/protected on interface methods (pre-Java 9)

1️⃣6️⃣ Access Modifiers & Abstract Methods

- Abstract methods cannot be private
- Can be protected or public

1️⃣7️⃣ Common MCQ Traps (🔥🔥🔥)

Statement	True/False
Private members inherited	❌
Protected accessible outside package without inheritance	❌
Default accessible in subclass outside package	❌
Public accessible everywhere	✅
Top-level class can be protected	❌

1️⃣8️⃣ Output-Based MCQ (🔥🔥)

```
java


// pkg1
class A {
    protected int x = 10;
}

// pkg2
class Test {
    public static void main(String[] args) {
        A a = new A();
        System.out.println(a.x);
    }
}
```

👉 Output:
❌ Compile-time error

1️⃣9️⃣ One-Line MCQ Answers (MEMORIZE)

- private → same class only

- default → same package
- protected → same package + subclass
- public → everywhere
- protected is mainly for inheritance

2️⃣0️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Most restrictive modifier?
   👉 private
2. Modifier accessible in subclass outside package?
   👉 protected
3. Default access means?
   👉 Same package
4. Top-level class can be?
   👉 public or default
5. Interface method access?
   👉 public
6. Can private method be overridden?
   👉 No
7. Which allows widest access?
   👉 public
8. Protected member accessible using parent reference in diff pkg?
   👉 No
9. Default member inherited in diff pkg?
   👉 No
10. Access modifier that supports inheritance best?
    👉 protected

✅ You are now MCQ-PROOF in Access Modifiers

What next to finish OOP MCQs?

🔥 **Object class methods (equals, hashCode, toString)**
🔥 **this vs super (mixed traps)**
🔥 **Final keyword (final class, method, variable)**

Tell me the next one and we'll close OOP completely 💪

> now static keyword(depth mcq concept)

Excellent 🔥
The `static` **keyword** is a **VERY HIGH-TRAP MCQ topic**. Examiners mix **memory, access rules, inheritance, polymorphism, blocks, and main()**.

Below is a **COMPLETE, deep, MCQ-PROOF guide**. Master this and static questions become easy marks.

⚡ `static` KEYWORD IN JAVA (DEEP MCQ CONCEPTS)

1️⃣ What is `static`?

Definition (Exam-ready):
`static` is a keyword used to make a **class-level member**, meaning it **belongs to the class, not to objects**.

👉 Keywords:

- Class level
- Single copy
- Shared among objects

2️⃣ Why `static` is Needed?

- Saves memory
- Common property for all objects
- Access without object

👉 MCQ trap:
❌ static does NOT improve performance directly

3️⃣ Static vs Non-Static (🔥 MUST MEMORIZE)

Static	Non-Static
Class level	Object level
One copy	Multiple copies
Access via class name	Access via object
Loaded once	Loaded per object

4️⃣ Static Variable (🔥🔥)

Example

```java
class Student {
    static int collegeCode = 101;
    int id;
}
```

MCQ Points:

- Single copy shared by all objects
- Stored in **Method Area / Metaspace**
- Initialized when class loads

👉 MCQ trap:

- Static variables exist even without object

5️⃣ Static Method (🔥🔥)

Example

```java
static void show() {
    System.out.println("Static method");
}
```

Rules (🔥 MCQ FAVORITE):

❌ Cannot access non-static members directly
❌ Cannot use **`this`** or **`super`**
✔️ Can access only static members

6️⃣ Why main() is static? (🔥🔥🔥)

```java
public static void main(String[] args)
```

👉 Reason:

- JVM calls main() **without creating object**

MCQ answer to remember:

> main() is static so JVM can call it directly

7️⃣ Static Block (🔥🔥🔥 VERY IMPORTANT)

Example

```
java

static {
    System.out.println("Static block");
}
```

MCQ Points:

- Executes when class is loaded
- Executes **before main()**
- Used for static initialization

👉 Order:

1. Static block
2. main()
3. Constructor

8️⃣ Multiple Static Blocks (MCQ)

- Executed in **top-to-bottom order**

9️⃣ Static Block vs Constructor (🔥)

Static Block	Constructor
Executes once	Executes per object
Class load time	Object creation
Cannot access instance members	Can access all

🔟 Static Keyword with Class (🔥 MCQ)

- Top-level class ❌ cannot be static
- Inner class ✅ can be static

1️⃣1️⃣ Static & Inheritance (🔥🔥)

✔️ Static members are inherited
❌ Static methods are NOT overridden

This is called **method hiding**

1️⃣2️⃣ Static Method Hiding (🔥🔥 MCQ TRAP)

```java
class A {
    static void show() { System.out.println("A"); }
}
class B extends A {
    static void show() { System.out.println("B"); }
}
A obj = new B();
obj.show();
```

👉 Output:

```css
A
```

Reason:

- Static method call depends on **reference type**

1️⃣3️⃣ Static Variable & Polymorphism (🔥)

- Static variables ❌ do NOT support polymorphism
- Resolved at compile time

1️⃣4️⃣ Static Import (MCQ)

```java
import static java.lang.Math.*;
```

👉 Allows calling:

```java
sqrt(16);
```

1️⃣5️⃣ Can Constructor be static? ❌

No, because constructor belongs to object creation.

1️⃣6️⃣ Can Static Method be Overloaded? ✅

YES.

1️⃣7️⃣ Can Static Method be Overridden? ❌

NO (method hiding occurs)

1️⃣8️⃣ Can Static Variables be Final? ✅

YES.

```
java

static final double PI = 3.14;
```

1️⃣9️⃣ Static & Memory (🔥 MCQ)

- Static members stored in **Method Area / Metaspace**
- Instance members stored in **Heap**

2️⃣0️⃣ Common MCQ Traps (🔥🔥🔥)

Statement	True/False
Static belongs to class	✅
Static method uses this	❌
Static block runs after main	❌
Static methods overridden	❌
Static variable shared	✅

2️⃣1️⃣ Output-Based MCQ (🔥🔥)

```
java

class Test {
    static {
        System.out.print("A ");
    }
    public static void main(String[] args) {
        System.out.print("B ");
    }
    Test() {
        System.out.print("C ");
    }
}
```

👉 Output:

```
css

A B
```

(Constructor not called)

2️⃣2️⃣ One-Line MCQ Answers (MEMORIZE)

- static = class level
- main() is static for JVM call
- Static block runs before main
- Static method cannot use this
- Static method is not polymorphic

2️⃣3️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. static belongs to?
   👉 Class
2. Number of static copies?
   👉 One
3. Static block executes when?
   👉 Class loading
4. main() is static because?
   👉 JVM calls it without object
5. Static method overridden?
   👉 No
6. Static method overloaded?
   👉 Yes
7. Static variable stored in?
   👉 Method Area
8. Can constructor be static?
   👉 No
9. Static inner class allowed?
   👉 Yes
10. Static supports polymorphism?
    👉 No

✅ You are now MCQ-PROOF in `static` keyword

What's left to COMPLETE Java OOP MCQs?

🔥 **final keyword (final variable, method, class traps)**
🔥 **Object class methods (equals, hashCode, toString)**
🔥 **Exception rules with overriding**

Tell me the next one and we'll finish strong 💪

> exception handling(deep mcq)

Perfect 🔥
Exception Handling is a **VERY HIGH-WEIGHT MCQ topic**, especially tricky because questions mix **hierarchy, checked vs unchecked, try–catch–finally, throw/throws, overriding rules, and output-based traps**.

Below is a **COMPLETE, deep, MCQ-PROOF guide**. After this, exception MCQs become scoring.

🚨 EXCEPTION HANDLING IN JAVA (DEEP MCQ LEVEL)

1️⃣ What is an Exception?

Definition (Exam-ready):
An **exception** is an **unexpected event** that occurs **during runtime** and disrupts the normal flow of the program.

👉 Keywords:

- Runtime error
- Abnormal condition
- Disrupts execution

2️⃣ Exception vs Error (🔥 VERY IMPORTANT)

Exception	Error
Recoverable	Not recoverable
Caused by program	Caused by system
Handled using try-catch	Cannot be handled
Example: IOException	Example: OutOfMemoryError

👉 MCQ trap:
❌ Errors are NOT meant to be caught

3️⃣ Exception Hierarchy (🔥🔥 MUST MEMORIZE)

```
php

Object
 └── Throwable
     ├── Exception
     │    ├── IOException (checked)
     │    ├── SQLException (checked)
     │    └── RuntimeException
     │         ├── NullPointerException
     │         ├── ArithmeticException
     │         └── ArrayIndexOutOfBoundsException
     └── Error
```

👉 MCQ:

- All exceptions inherit from **`Throwable`**

4️⃣ Types of Exceptions (🔥🔥)

1. Checked Exceptions

- Checked at **compile time**
- Must be handled or declared

Examples:

- IOException
- SQLException
- FileNotFoundException

---

2. Unchecked Exceptions

- Checked at **runtime**
- Subclasses of **`RuntimeException`**

Examples:

- NullPointerException
- ArithmeticException
- NumberFormatException

👉 MCQ TRAP:
❌ RuntimeException is NOT checked
❌ Exception ≠ RuntimeException

5️⃣ Keywords Used in Exception Handling (🔥)

- `try`
- `catch`
- `finally`
- `throw`
- `throws`

6️⃣ try–catch Block (🔥🔥)

```
java

try {
    int a = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error");
}
```

Rules:

- try must be followed by catch or finally
- Multiple catch blocks allowed

7️⃣ Multiple catch Blocks (🔥 MCQ)

```
java

try {
    ...
} catch (ArithmeticException e) {
} catch (Exception e) {
}
```

👉 Rule:

- Child exception first
- Parent exception last

❌ Reverse order → Compile-time error

8️⃣ finally Block (🔥🔥🔥 VERY IMPORTANT)

Rules:

- Executes **always**
- Used for cleanup (closing resources)

```java
finally {
    System.out.println("Done");
}
```

👉 MCQ traps:

- finally executes even if exception not handled
- finally does NOT execute only when JVM crashes (`System.exit()`)

9️⃣ try without catch? (🔥 MCQ)

YES — if `finally` is present.

```java
try {
} finally {
}
```

🔟 catch without try? ❌

Not allowed.

1️⃣1️⃣ Nested try–catch (MCQ)

Allowed.

1️⃣2️⃣ throw Keyword (🔥🔥)

Purpose:

- Explicitly throw an exception

```java
throw new ArithmeticException();
```

Rules:

- Used inside method
- Throws **one exception at a time**

1️⃣3️⃣ throws Keyword (🔥🔥)

Purpose:

- Declares exception to caller

```
java

void read() throws IOException {}
```

👉 MCQ trap:

- throws does NOT throw exception
- throws can declare multiple exceptions

1️⃣4️⃣ throw vs throws (🔥🔥)

throw	throws
Used inside method	Used in method signature
Throws single exception	Declares multiple
Creates exception object	No object creation

1️⃣5️⃣ Checked Exception Rules (🔥🔥 MCQ FAVORITE)

Checked exception must be:
✔️ handled using try–catch
OR
✔️ declared using throws

Otherwise → ❌ Compile-time error

1️⃣6️⃣ Method Overriding & Exceptions (🔥🔥🔥 VERY IMPORTANT)

Rules:

- Child method CANNOT throw broader checked exception
- Can throw same or subclass
- Can throw unchecked exception freely

👉 MCQ TRAP:
Unchecked exceptions have **no restriction**

1️⃣7️⃣ Custom Exception (User-Defined) (🔥)

```java
class MyException extends Exception {}
```

👉 MCQ:

- Can extend Exception or RuntimeException

1️⃣8️⃣ finally vs return (🔥 MCQ TRAP)

```java
try {
    return 10;
} finally {
    return 20;
}
```

👉 Output:

```
20
```

Reason:

- finally overrides return

1️⃣9️⃣ Exception Propagation (🔥)

- Unhandled exception moves up call stack
- Only unchecked exceptions propagate automatically

2️⃣0️⃣ Common Runtime Exceptions (🔥 MCQ)

- NullPointerException
- ArithmeticException
- ArrayIndexOutOfBoundsException
- ClassCastException
- NumberFormatException

2️⃣1️⃣ Can We Catch Multiple Exceptions at Once? (Java 7+)

YES.

```
java

catch (IOException | SQLException e) {}
```

👉 MCQ:

- Exception types must be unrelated

2️⃣2️⃣ Common MCQ Traps Table (🔥🔥🔥)

Statement	True / False
Exception occurs at compile time	❌
Checked exception mandatory handling	✅
finally always executes	✅ (except JVM exit)
throw creates exception object	✅
throws handles exception	❌

2️⃣3️⃣ Output-Based MCQ (🔥🔥)

```
java

try {
    System.out.print("A ");
    int x = 10 / 0;
    System.out.print("B ");
} catch (Exception e) {
    System.out.print("C ");
} finally {
    System.out.print("D ");
}
```

👉 Output:

```
mathematica

A C D
```

2️⃣4️⃣ One-Line MCQ Answers (MEMORIZE)

- Checked → compile time
- Unchecked → runtime
- Throwable is root class
- finally used for cleanup
- throw throws exception

- throws declares exception

2️⃣5️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. Root class of exception hierarchy?
   👉 Throwable
2. Which is unchecked?
   👉 NullPointerException
3. finally block executes when?
   👉 Always
4. Keyword to explicitly throw exception?
   👉 throw
5. Keyword to declare exception?
   👉 throws
6. Checked exception handling mandatory?
   👉 Yes
7. Can finally override return?
   👉 Yes
8. Multiple catch order?
   👉 Child first
9. Can we define our own exception?
   👉 Yes
10. RuntimeException is?
    👉 Unchecked

✅ You are now MCQ-PROOF in Exception Handling

🔥 Final OOP MCQ topics left:

- **final keyword (final variable, method, class traps)**
- **Object class methods (equals, hashCode, toString MCQs)**
- **this vs super (mixed tricky MCQs)**

Tell me the next one and we'll COMPLETE Java OOP 💪

> Relationship Between Classes(deep mcq concepts)

Excellent 🔥
Relationship Between Classes is a **classic MCQ topic** because questions mix **IS-A vs HAS-A,**

inheritance vs association, aggregation vs composition, UML ideas, and real-life examples.

Below is a **COMPLETE, deep, MCQ-PROOF guide**. After this, you won't confuse relationships again.

🔗 RELATIONSHIP BETWEEN CLASSES (DEEP MCQ CONCEPTS)

1️⃣ What is a Relationship Between Classes?

Definition (Exam-ready):
A **relationship between classes** defines **how classes are connected and interact with each other** in an object-oriented system.

2️⃣ Main Types of Relationships in Java (🔥 MUST KNOW)

1. **IS-A relationship**
2. **HAS-A relationship**
3. **Association**
4. **Aggregation**
5. **Composition**
6. **Dependency** (less common, but MCQs appear)

3️⃣ IS-A Relationship (Inheritance) (🔥🔥 VERY IMPORTANT)

Meaning:

- One class **IS A** type of another class
- Implemented using **inheritance**

```
java

class Animal {}
class Dog extends Animal {}
```

👉 MCQ:

- Dog IS-A Animal ✅
- Inheritance represents IS-A

MCQ Traps:

❌ IS-A ≠ HAS-A
❌ IS-A ≠ uses-a

4️⃣ HAS-A Relationship (🔥🔥 VERY IMPORTANT)

Meaning:

- One class **HAS A** reference of another class
- Implemented using **object reference**

```
java

class Engine {}
class Car {
    Engine e = new Engine();
}
```

👉 MCQ:

- Car HAS-A Engine
- HAS-A promotes **code reuse without inheritance**

5️⃣ IS-A vs HAS-A (🔥🔥🔥 VERY COMMON MCQ)

IS-A	HAS-A
Inheritance	Association
`extends`	Object reference
Strong coupling	Loose coupling
Dog IS-A Animal	Car HAS-A Engine

👉 MCQ trap:
Prefer HAS-A over IS-A for flexibility

6️⃣ Association (🔥 MCQ)

Definition:
Association is a **general relationship** where one class is connected to another.

```
java

class Teacher {}
class Student {
    Teacher t;
}
```

👉 MCQ:

- Association can be one-to-one, one-to-many, many-to-many

7️⃣ Types of Association

1. One-to-One
2. One-to-Many
3. Many-to-One
4. Many-to-Many

👉 MCQ:

- Association does NOT imply ownership

8️⃣ Aggregation (🔥🔥 VERY IMPORTANT)

Definition:

Aggregation is a **weak HAS-A relationship** where:

- Child object **can exist independently** of parent

```
java

class Department {}
class University {
    Department d;
}
```

👉 MCQ:

- University HAS-A Department
- Department can exist without University

9️⃣ Composition (🔥🔥 VERY IMPORTANT)

Definition:

Composition is a **strong HAS-A relationship** where:

- Child object **cannot exist without parent**

```
java

class Heart {}
class Human {
    Heart h = new Heart();
}
```

👉 MCQ:

- Human HAS-A Heart

- Heart cannot exist without Human

🔟 Aggregation vs Composition (🔥🔥🔥 MOST CONFUSING MCQ)

Aggregation	Composition
Weak HAS-A	Strong HAS-A
Child independent	Child dependent
Uses external object	Creates object internally
Example: University–Department	Example: Human–Heart

1️⃣1️⃣ Ownership Concept (MCQ)

- Aggregation → No ownership
- Composition → Ownership exists

1️⃣2️⃣ UML Representation (MCQ Concept)

- Association → Simple line
- Aggregation → Hollow diamond
- Composition → Filled diamond

👉 Even if UML not asked, concept appears in MCQs.

1️⃣3️⃣ Dependency Relationship (🔥 MCQ)

Definition:
A class **depends on** another class temporarily (method parameter or local variable).

```
java


class Printer {}
class Computer {
    void print(Printer p) {}
}
```

👉 MCQ:

- Dependency is a **uses-a relationship**
- Short-term relationship

1️⃣4️⃣ Relationship Strength Order (🔥 MCQ)

From weakest to strongest:

```
mathematica

Dependency → Association → Aggregation → Composition
```

1️⃣5️⃣ Relationship & Coupling (MCQ)

- IS-A → Tight coupling
- HAS-A → Loose coupling

👉 MCQ:
Loose coupling is preferred

1️⃣6️⃣ Can Inheritance Replace HAS-A? (MCQ)

❌ NO.

👉 Reason:

- Inheritance misused leads to poor design

1️⃣7️⃣ Real-Life MCQ Examples (🔥)

Relationship	Example
IS-A	Dog–Animal
HAS-A	Car–Engine
Aggregation	Library–Books
Composition	House–Rooms
Dependency	Order–Payment

1️⃣8️⃣ Common MCQ Traps (🔥🔥🔥)

Statement	True/False
Inheritance is HAS-A	❌
Aggregation allows independent child	✅
Composition allows independent child	❌
HAS-A uses object reference	✅
Dependency is permanent	❌

1️⃣9️⃣ One-Line MCQ Answers (MEMORIZE)

- IS-A → inheritance
- HAS-A → object reference
- Aggregation → weak HAS-A
- Composition → strong HAS-A
- Dependency → uses-a

2️⃣0️⃣ Output-Based MCQ (Conceptual)

```
java


class A {}
class B {
    A a;
}
```

👉 Relationship?
HAS-A (Association)

2️⃣1️⃣ 10 EXAM-LEVEL MCQs (With Answers)

1. IS-A relationship achieved using?
   👉 Inheritance
2. HAS-A relationship achieved using?
   👉 Object reference
3. Weak HAS-A is called?
   👉 Aggregation
4. Strong HAS-A is called?
   👉 Composition
5. Which allows child independent existence?
   👉 Aggregation
6. Which is uses-a relationship?
   👉 Dependency
7. Car–Engine is?
   👉 HAS-A
8. Dog–Animal is?
   👉 IS-A
9. Composition implies?
   👉 Ownership

10. Loose coupling preferred using?
    👉 HAS-A

✅ You are now MCQ-PROOF in Class Relationships

🎯 Final OOP MCQ topics to finish everything:

🔥 **final keyword (deep traps)**
🔥 **Object class methods (equals, hashCode, toString)**
🔥 **Packages & imports (MCQ concepts)**

Tell me the next topic and we'll COMPLETE Java OOP MCQs 💪